

Propguru AI Agent Platform

Product Requirements Document (PRD)

Version: 2.0

Prepared by: AI Platform Team

Audience: Propguru Leadership, Product Owner

Status: Ready for Review

Scope: This PRD covers the AI Agent Platform and its first deployed use case — Property Acquisition Evaluation.

1. Executive Summary

Propguru operates across several business workflows that share a common problem: they are knowledge-intensive, time-consuming, and require expert judgment that is hard to scale. Property acquisition evaluation is the first and most acute of these. But the opportunity is broader.

This document defines requirements for a **modular AI Agent Platform** built for Propguru's operations. The platform is not built to automate a single workflow — it is built to automate *any* structured workflow within Propguru's business by combining:

- **AI agents** that execute domain-specific reasoning tasks
- **A tool registry** that connects agents to Propguru's data sources
- **A HITL (Human-in-the-Loop) workflow** that keeps analysts in control of every outcome
- **Automated quality gates** that verify AI output before it reaches a human
- **A configurable runtime** where models, prompts, and agent behavior are driven by configuration — no code changes required to switch providers or adjust routing

The platform is **model-agnostic** (supports OpenAI, Anthropic, and others via YAML config), **tool-agnostic** (new data connectors are registered as tools without touching the orchestration layer), and **use-case agnostic** (new workflows are added by writing agent configs and domain tools — the platform infrastructure stays unchanged).

The first use case — Property Acquisition Evaluation — automates:

- Scoring 30 property criteria from structured data and AI estimation
- Fetching market comparables and computing a data-backed acquisition price
- Running two-phase quality verification (deterministic + LLM-as-judge)
- Surfacing results to the analyst for review, refinement, and approval

Core value delivered by this use case:

- Evaluation time: **2–3 hours → 3–5 minutes**
- Consistency: same 30 criteria, same formula, every deal
- Analyst leverage: review and refine instead of build from scratch
- Audit trail: every score, price computation, and analyst decision recorded

2. Platform Overview

2.1 What the Platform Provides

The AI Agent Platform is the shared foundation that all Propguru use cases run on. It provides:

Capability	What It Does
Agent Registry	YAML-defined agents stored in a database; activated on demand
Supervisor-Worker Orchestration	LangGraph-based multi-agent graphs; supervisor routes work to specialist workers
Tool Registry	Domain tools (data connectors, calculators, proposal writers) registered as callable functions
Async Task Execution	Celery workers execute agent pipelines without blocking the UI
HITL Workflow	Every AI output becomes a proposal; humans approve, reject, or refine before any state changes
Verification Loop	Automated quality gates run after AI completes; failures trigger self-correction retries
Audit Trail	Every agent run, tool call, proposal, and analyst decision is persisted
Model Agnosticism	<code>provider: openai</code> / <code>provider: anthropic</code> in YAML — no code change to switch models
Observability	Token usage, cost, duration, LangSmith traces, and OpenTelemetry spans per run

2.2 How Use Cases Are Built on the Platform

A new Propguru use case is added by:

1. **Domain schema** — New PostgreSQL tables for domain entities (e.g., `propguru_deals`, `propguru_properties`)
2. **Domain tools** — Python functions registered as tools that agents can call (e.g., `propguru_get_deal`, `propguru_calculate_price`)
3. **Agent configs** — YAML files that define agent behavior: model, system prompt, tools, feature flags
4. **API routes** — FastAPI routes that trigger agent runs and expose domain data
5. **UI pages** — Jinja2 templates or nextjs app for the domain's analyst interface

Nothing in the platform's core changes — the orchestration engine, HITL pattern, verification loop, task queue, and model integration are reused across every use case.

2.3 Current Use Cases

Domain	Use Case	Status
Propguru	Property Acquisition Evaluation	Live (POC)
Propguru	<i>(future use cases — see roadmap)</i>	Planned

2.4 Platform Design Principles

Model-Agnostic: No AI provider is hardcoded. Each agent declares `provider` and `name` in its YAML. Switching from GPT-4o to Claude Sonnet is a one-line config change — no code deployment.

Tool-Agnostic: Agents call tools by name. The tool implementation (API call, DB query, calculation) is decoupled from the agent. Swapping a data source means updating the tool implementation, not the agent.

Human-In-The-Loop by Design: AI output is always a *proposal*. No deal stage changes, no price is set, and no action is taken until a human explicitly approves. HITL is enforced at the platform level — it cannot be bypassed by individual use cases.

Self-Correcting Quality Gates: Platform-level verification nodes run after AI completes. If output fails quality checks, the system retries automatically with targeted feedback. Analysts only see output that has passed verification.

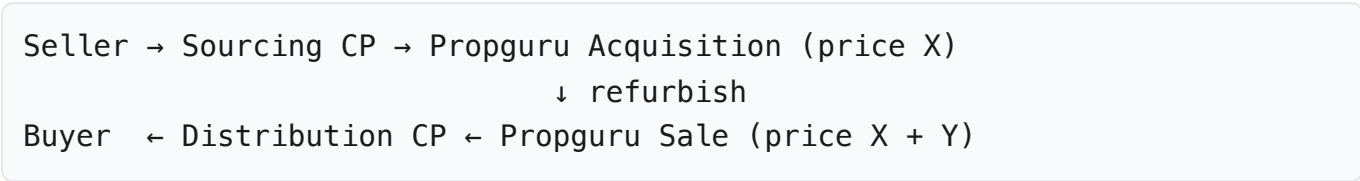
Configuration-Driven Behavior: System prompts, model selection, tool availability, routing rules, feature flags, and quality thresholds are all in YAML and environment config. Adding a new tool to an agent, changing its model, or enabling a feature flag requires no code deployment.

Separation of Concerns: Platform code (orchestration, HITL, task queue) is kept separate from domain code (Propguru tools, Propguru UI). Adding a new domain does not touch platform code.

3. Propguru Business Context

3.1 Business Model

Propguru acquires residential properties, refurbishes, and sells them through Channel Partners (CPs). The acquisition margin depends critically on getting the entry price right.



Every workflow that informs price X, deal quality, or partner management is a candidate for AI automation on this platform.

3.2 Workflows Identified for Platform Support

Workflow	Current State	AI Platform Opportunity
Property Acquisition Evaluation	Manual, 2–3 hrs/deal	Automated via 4-agent pipeline ← <i>This PRD</i>
Channel Partner Lead Scoring	Informal gut-feel	Score CP quality by historical deal outcomes
Refurbishment Estimation	Manual cost estimate	AI-assisted cost range from property attributes
Listing Price Optimization	Analyst + market feel	Price optimization agent using market velocity data
Portfolio Health Monitoring	Monthly manual report	Real-time portfolio agent with threshold alerts

The platform is designed so each of these can be onboarded as a new use case without changing the platform infrastructure.

4. Use Case: Property Acquisition Evaluation

This section covers the full requirements for the first deployed use case.

4.1 Problem Statement

When a new property deal comes in, the current evaluation process:

1. A CP submits a lead with basic property details

2. A deal manager manually looks up comparable transactions in the locality
3. The deal manager scores the property informally — notes on floor, facing, nearby schools, amenities — and arrives at a gut-feel price
4. The evaluation is reviewed (if time permits) by a senior analyst
5. A price recommendation is communicated back to the CP

Pain points:

Pain Point	Business Impact
No standard scoring rubric	Two analysts evaluate the same property differently
Manual market comp lookup	30–60 minutes per deal; data may be stale
Price justification is informal	Hard to defend pricing in negotiation or to investors
No audit trail on pricing rationale	Compliance and accountability gaps
Volume doesn't scale	Evaluation is the bottleneck when deal flow increases
Analyst time spent on data gathering, not judgment	High-value human time on low-value tasks

4.2 Solution

The evaluation use case automates the mechanical part of analysis using a 4-agent AI pipeline and surfaces results for analyst review via HITL. The analyst's role shifts from *building the evaluation* to *reviewing and approving it*.

4.3 User Personas

Deal Manager (Primary – Initiates Evaluation)

Manages the deal pipeline, communicates with CPs, decides which leads to advance.

Pain: Spends 2–3 hours per deal on data gathering before it can go to the analyst.

Platform gives them: One click triggers the full evaluation. Status updates in real time. Re-evaluation is easy if needed.

Key question answered: *"Is this property worth buying, and at what price?"*

Acquisition Analyst (Primary – Reviews and Approves)

Reviews AI evaluations, applies domain knowledge, makes the final call on price.

Pain: Reviews are inconsistent in format. Critical information is sometimes missing. Hard to compare evaluations across deals.

Platform gives them: Standardised report for every deal — same 30 criteria, same format, every time. Transparent price derivation. Inline refinement canvas for score adjustments. Quality flags when the AI had issues.

Key question answered: *"Is this evaluation correct, and am I comfortable approving this price?"*

Operations Lead (Secondary – Manages Configuration)

Oversees system configuration, CP relationships, and master data.

Platform gives them: Central master data management for properties, CPs, and market comps. Ability to update criteria weights. Audit logs for all agent actions and analyst decisions.

Key question answered: *"Is the platform configured correctly, and is evaluation quality improving?"*

Channel Partner (Indirect – Submits Leads)

External sourcing partner who brings deals to Propguru.

Current state: Submits leads informally (call, email). Follows up manually on status.

Future integration (roadmap): Deal submissions via CP portal → automatic intake into evaluation pipeline → status updates back to CP

4.4 User Scenarios

Scenario 1: Standard Lead-to-Approval

1. Deal Manager logs the property (or auto-ingest from CP portal — roadmap)
2. Deal Manager opens Deals Pipeline, sees new lead as "Lead"
3. Deal Manager clicks ► **Evaluate** — AI pipeline starts
4. Status changes to "Running" (spinner badge on the deal row)
5. In 3–5 minutes, pipeline completes:
 - Data collector scores property attributes (floor, facing, age, etc.)
 - Market analyst fetches locality market rate, computes base price
 - Scorer estimates amenity and location scores
 - Evaluator computes: base ₹85L + 26% premium = ₹1.07 Cr recommended
 - Quality gates pass → evaluation reaches analyst inbox
6. Analyst reviews → notices jogging track scored 0 but was seen on site visit → opens **Refine** → price adjusts to ₹1.09 Cr
7. Analyst clicks **Approve** → deal advances to "Evaluation Done"
8. Deal Manager sees updated status, communicates price to CP

Time: ~5 min AI + ~10 min analyst review vs. 2–3 hours today

Scenario 2: AI Self-Correction via Quality Gate

1. Evaluator completes scoring but misses 4 criteria (26/30 — below threshold)
2. **Phase 1 Code Grader** detects: **COVERAGE: 26/30** criteria scored. Missing: **CRIT-007, CRIT-019, CRIT-027, CRIT-029**
3. System automatically:
 - Dismisses the incomplete proposal from analyst inbox
 - Injects corrective feedback to the AI with specific missing criterion IDs
 - Resets evaluation to draft and re-runs the evaluator
4. Second attempt: all 30 criteria scored; model grader scores reasoning 7.2/10 (passes ≥ 6.0)
5. Evaluation reaches analyst inbox — complete and verified

Analyst never sees the failed evaluation. They only see the corrected one.

Scenario 3: Analyst Refinement

1. AI recommends ₹3.8 Cr for a Bandra West property
 2. Analyst knows from recent off-market transaction that market rate should be ₹37,500/sqft (AI used ₹35,000 from stored comp)
 3. Analyst opens **Refine Canvas** → types correction → base price recalculates → ₹3.98 Cr recommended
 4. Analyst also confirms pool (CRIT-001 = 1) → final price ₹4.02 Cr
 5. Analyst approves with notes documenting the market rate update
 6. System records: original AI price, analyst-adjusted price, reasoning — full audit trail
-

Scenario 4: Escalation After Quality Gate Failure

1. Evaluator uses fallback market rate for unknown locality → sets confidence "high" despite low coverage
 2. Phase 1 Code Grader **FAIL: CONFIDENCE_MISMATCH + COVERAGE: 24/30**
 3. Evaluator retries after feedback (attempt 2) — improves to 28 criteria but keeps "high" confidence
 4. After 2 retries (max), escalates to analyst inbox with flags: **COVERAGE, CONFIDENCE_MISMATCH**
 5. Analyst sees ⚠️ **GRADER FLAGGED** badge, manually refines confidence to "low", adds notes
 6. Approves with flagged context documented
-

5. Functional Requirements

5.1 Platform-Level Requirements

These requirements apply to the platform and benefit all current and future use cases.

ID	Requirement	Priority
PFR-001	Platform shall maintain an agent registry: YAML-defined agents stored in the DB, activatable without code changes	P0
PFR-002	Platform shall execute agent pipelines asynchronously via a task queue (Celery + Redis)	P0
PFR-003	Platform shall support supervisor-worker multi-agent graphs where a supervisor LLM routes work to specialist worker agents	P0
PFR-004	Platform shall support configuring AI models per agent via YAML (provider, name, temperature, max_tokens) with no code changes	P0
PFR-005	Platform shall maintain a tool registry: callable Python functions registered by name, available to agents via YAML config	P0
PFR-006	Platform shall implement a HITL workflow: every AI output becomes an AgentAction proposal requiring explicit analyst approve/reject/refine before any state mutation	P0
PFR-007	Platform shall implement a verification loop: configurable verification node runs after designated worker; failed checks inject feedback and retry automatically	P0
PFR-008	Platform shall record all agent runs with: status, input, output,	P0

ID	Requirement	Priority
	error, token usage, cost, duration, LangSmith trace URL	
PFR-009	Platform shall persist all AgentAction proposals and analyst decisions immutably for audit	P0
PFR-010	Platform shall provide consistent API auth: X-API-Key header (POC); replaceable with OAuth2/SSO middleware for production	P0
PFR-011	Adding a new use case shall require only: domain DB tables, domain tools, YAML agent configs, and API/UI routes — no changes to platform core	P0
PFR-012	Platform shall expose agent run status and output via REST API, queryable by run ID and agent name	P1
PFR-013	Platform shall support feature flags per agent via YAML (feature_flags block), enabling/disabling capabilities without code changes	P1

5.2 Propguru Evaluation – Deal Pipeline

ID	Requirement	Priority
FR-001	System shall maintain a deal pipeline: Lead → Evaluation Pending → Evaluation Done → Agreement Signed → Listed → Sold / Lost	P0
FR-002	Each deal shall be linked to a property record and optionally a channel partner	P0
FR-003	Deal Manager shall be able to advance, revert, or close a deal manually via UI	P0
FR-004	Deals in Lead or Evaluation Pending stage can be submitted for AI evaluation	P0
FR-005	System shall prevent concurrent evaluations on the same deal	P0
FR-006	If evaluation fails (all retries exhausted), deal shall automatically return to Lead stage so re-evaluation can be triggered	P0

5.3 Propguru Evaluation – AI Pipeline

ID	Requirement	Priority
FR-010	System shall run a 4-stage sequential AI evaluation pipeline: Data Collection → Market Analysis → Scoring → Evaluation	P0
FR-011	Data Collection stage shall score property attribute criteria from property master data using domain-specific scoring rules	P0
FR-012	Market Analysis stage shall fetch locality-level market comparables and compute base acquisition price	P0
FR-013	Market Analysis shall fall back to a configurable default rate if no locality comp exists, and mark confidence accordingly	P0
FR-014	Scoring stage shall estimate amenity (boolean) and location proximity (distance-based) criteria	P0
FR-015	Evaluation stage shall compute recommended price: $\text{base_price} \times (1 + \text{score_factor} \times \text{MAX_PREMIUM_PCT})$	P0
FR-016	Evaluation stage shall determine confidence level (high/medium/low) based on coverage and score factor	P0
FR-017	Evaluation stage shall produce human-readable reasoning text for analyst review	P0
FR-018	Evaluation stage shall submit the result as a HITL proposal via	P0

ID	Requirement	Priority
	the platform HITL workflow	
FR-019	Entire pipeline shall complete within 5 minutes for a standard evaluation	P1

5.4 Propguru Evaluation – Scoring Model

ID	Requirement	Priority
FR-020	System shall maintain 30 evaluation criteria across 4 categories: Amenity (10), Location (10), Property (5), Society (5)	P0
FR-021	Each criterion shall have a weight (importance) configurable by the Operations Lead via UI	P0
FR-022	Three scoring types shall be supported: <code>boolean</code> (0 or 1), <code>scale_1_5</code> (1–5), <code>proximity_km</code> (0–5 km)	P0
FR-023	Score normalization to [0,1] shall be applied per type before weighted average	P0
FR-024	Boolean criteria shall only accept 0.0 or 1.0 — any other value is rejected at the API layer	P0
FR-025	Criteria shall be activatable/deactivatable individually without code changes	P1
FR-026	Criteria weights shall be updatable via the master data UI	P1

5.5 Propguru Evaluation – Quality Verification

These requirements use the platform verification loop (PFR-007) configured for the evaluation use case.

ID	Requirement	Priority
FR-030	Phase 1 Code Grader shall run automatically after every evaluator completion	P0
FR-031	Code Grader shall enforce: coverage $\geq 28/30$, boolean validity, price within $\pm 50\%$ of base, confidence calibration, no category zero-out	P0
FR-032	On Code Grader FAIL, system shall inject corrective feedback, dismiss pending proposal, reset to draft, and re-run evaluator	P0
FR-033	Code Grader retry limit shall be configurable (default: 2 retries)	P1
FR-034	On Code Grader FAIL after max retries, system shall escalate to analyst inbox with grader flags marked	P0
FR-035	Phase 2 Model Grader shall run after Phase 1 passes, grading reasoning quality on 4 rubric criteria	P1
FR-036	Model Grader shall require weighted average $\geq 6.0/10$ across: reasoning coherence, price justification, market alignment, analyst guidance	P1
FR-037	On Model Grader FAIL, system shall inject per-criterion feedback and allow one evaluator retry	P1
FR-038	If either grader encounters infrastructure errors, evaluation shall pass through (fail-open) with flag recorded	P0

ID	Requirement	Priority
FR-039	All grader flags, retry counts, and verdicts shall be persisted to the evaluation report	P0

5.6 Propguru Evaluation – HITL Workflow

These requirements implement the platform HITL workflow (PFR-006) for the evaluation use case.

ID	Requirement	Priority
FR-040	Every AI evaluation shall produce a HITL proposal requiring analyst action before any deal stage changes	P0
FR-041	Analyst shall be able to Approve, Reject, or Refine an evaluation	P0
FR-042	Approval shall set the final acquisition price, advance deal to Evaluation Done, and record approval timestamp + analyst name	P0
FR-043	Rejection shall return the deal to Evaluation Pending	P0
FR-044	Refinement shall allow natural language instructions to update individual scores and recompute price	P0
FR-045	Scores modified by analyst shall be tracked (source: "analyst") separately from AI scores (source: "agent")	P0
FR-046	Each refinement cycle shall increment the report version number	P1

5.7 Propguru Evaluation – Refinement Canvas

ID	Requirement	Priority
FR-050	Refinement canvas shall display top 10 criteria by weight with current scores and accept natural language update instructions	P0
FR-051	Canvas shall show category subtotals (weighted % for each of the 4 categories)	P0
FR-052	Canvas shall show live price recalculation as scores change	P0
FR-053	Canvas shall show original AI recommendation alongside current refined price	P1

5.8 Propguru Evaluation – Master Data

ID	Requirement	Priority
FR-060	System shall maintain property master: address, locality, city, type, dimensions, floor, facing, age, GPS coordinates	P0
FR-061	System shall maintain channel partner records: name, type, commission rate, contact details	P0
FR-062	System shall maintain market comparable data per locality: avg/min/max price per sqft, 6-month trend, transaction volume, data source	P0
FR-063	Operations Lead shall be able to update property, CP, and market comp records via UI	P0
FR-064	In production, market comp data shall be refreshable from live data providers without code changes	P1

5.9 Reporting and Audit

ID	Requirement	Priority
FR-070	Every evaluation report shall record: all 30 scores, score source, price derivation, AI reasoning, analyst notes, grader flags, approval details	P0
FR-071	Full history of agent actions (proposals, dismissals, refinements) shall be retained and queryable	P0
FR-072	Evaluation reports shall be accessible via REST API for downstream integration (BI dashboards, CRM)	P1

6. Non-Functional Requirements

6.1 Performance

ID	Requirement
NFR-001	Full 4-agent evaluation pipeline shall complete within 5 minutes under normal conditions
NFR-002	UI page load times shall be under 2 seconds for the deal pipeline view
NFR-003	Refinement canvas price recalculation shall respond within 30 seconds of AI instruction
NFR-004	Platform shall support at least 10 concurrent evaluations without degradation

6.2 Reliability

ID	Requirement
NFR-010	Agent tasks shall be retried automatically on transient failures (up to 2 retries, configurable)
NFR-011	If the quality grader fails due to infrastructure issues, the pipeline shall continue (fail-open)
NFR-012	Domain entity state shall always reflect the true pipeline status — a failed run shall never leave a deal permanently stuck
NFR-013	Database transactions shall be atomic — partial writes are never committed

6.3 Security

ID	Requirement
NFR-020	All API endpoints shall require authentication (POC: API key; production: bearer token / SSO)
NFR-021	All AI model API keys shall be stored as environment variables, never in code or config files
NFR-022	Audit trail records shall be immutable — analyst actions are appended, not overwritten
NFR-023	In production, role-based access shall restrict which analysts can approve evaluations above a configured price threshold

6.4 Model and Tool Agnosticism

ID	Requirement
NFR-030	The AI model used for each agent shall be configurable via YAML without code changes
NFR-031	The platform shall support OpenAI and Anthropic Claude models per agent, switchable via config
NFR-032	Domain business logic (criteria weights, normalization formulas, price formula) shall be separated from model selection
NFR-033	Adding a new data source connector shall require only a new tool implementation — no changes to the agent or orchestration layer
NFR-034	If a model provider is unavailable, the system shall surface a clear error rather than silently producing incorrect results

6.5 Maintainability

ID	Requirement
NFR-040	Evaluation criteria (weights, scoring types, descriptions) shall be updatable from the UI without code changes
NFR-041	New agent capabilities shall be addable via YAML configuration — no code deployment required for routing changes
NFR-042	Quality gate thresholds shall be configurable via environment variables and feature flags
NFR-043	New use cases shall be onboardable without modifying platform core code

6.6 Observability

ID	Requirement
NFR-050	All agent runs shall be traceable: input, output, token usage, cost, duration
NFR-051	Failed evaluations shall log full error with stack trace and retry count
NFR-052	In production, LangSmith tracing shall capture the full multi-agent reasoning chain

7. Integration with Existing Systems

7.1 POC vs Production Data

The POC uses seed data. In production, each data source is replaced by integration with live systems:

Data Source	POC (Seed)	Production Integration
Property records	10 seeded properties (PROP-001–010)	Read from Propguru property database / CRM
Market comparables	5 manually seeded locality comps	API call to Housing.com, MagicBricks, or internal comp DB
Channel partners	2 seeded partners (CP-001–002)	Sync from CRM / partner portal
Deal intake	Manual entry via API	Auto-ingest from lead form / CP portal / WhatsApp bot

The platform's tool implementations are the only layer that changes. The AI pipeline, scoring model, quality gates, and HITL workflow remain identical.

7.2 Authentication

- POC: API key (X-API-Key header)
- Production: Propguru's existing auth (OAuth2 / SSO) via replaceable middleware

7.3 Notifications

- POC: Status visible in UI
- Production: Webhook / email / Slack when evaluation completes and proposal is ready for analyst

7.4 Downstream Systems

Approved evaluations expose structured data (recommended price, score breakdown, confidence, analyst notes) via REST API for:

- BI dashboards
 - CRM deal record updates
 - CP portal status updates
-

Appendix: Feature Requests and Improvements Backlog

Purpose: This section captures feature requests and improvement ideas raised by the product owner that have not yet been scoped, designed, or prioritised. Items here are for review and discussion — nothing below is committed or scheduled. They will be moved into the roadmap once reviewed and accepted.

A1. City-wise Macro and Micro Market Criteria (Global Rule Layer)

Request: Introduce a global set of evaluation rules that apply at the city or micro-market level, layered on top of the existing 30-criteria property-level scoring.

Context: The current evaluation model scores each property in isolation using criteria with fixed weights. In practice, the desirability of a property is heavily influenced by the broader market it sits in — a 3BHK in a premium IT corridor in Bengaluru scores differently in context than an identical property in a tier-3 city, even if the 30 property-level criteria are identical.

What is being asked for:

- A **macro market layer** that captures city-level indicators: economic growth profile, employment base, infrastructure investment pipeline, population inflow trend
 - A **micro market layer** that captures locality-level indicators within a city: sub-market demand velocity, upcoming supply pipeline, historical price appreciation, investor vs end-user mix
-

A2. Price Range Output: Acquisition Price (X) and Selling Price (Y)

Request: Replace the current single recommended price output with a **price range**: a lower bound (acquisition / buy price X) and an upper bound (selling price Y), where the spread X→Y represents the deal's profit profile.

Context: The current model outputs one number — a recommended acquisition price derived from market comps and property scoring. This is useful but incomplete. Propguru's business objective is to **maximise the margin between acquisition cost and selling price**. To evaluate a deal properly, the analyst needs both endpoints of that range, not just the entry price.

What is being asked for:

- **X — Acquisition price (lower bound):** The price Propguru should target when buying the property from the seller. Derived from market comps, property scoring, and negotiation headroom.
- **Y — Selling price (upper bound):** The price Propguru can realistically achieve when selling the property (post-refurbishment). Derived from market demand, comparable listed prices, and expected refurbishment uplift.
- **X-Y = Deal profit profile:** The spread. A wider spread indicates a more attractive deal. The platform should help the analyst understand whether a deal is worth pursuing based on this spread, not just whether the acquisition price is fair.